information

MDPI

Article

Using Large Language Model to Fill in Web Forms to Support Automated Web Application Testing

Feng-Kai Chen [1], Chien-Hung Liu [2] and Shingchern D. You [2,*]

[1] Qnap, New Taipei City 221, Taiwan; timtim8361@gmail.com
[2] Department of Computer Science and Information Engineering, National Taipei University of Technology, Taipei 106, Taiwan; cliu@ntut.edu.tw
* Correspondence: scyou@ntut.edu.tw; Tel.: +886-2-2771-2171 (ext. 4234)

Abstract: Web applications, widely used by enterprises for business services, require extensive testing to ensure functionality. Performing form testing with random input data often takes a long time to complete. Previously, we introduced a model for automated testing of web applications using reinforcement learning. The model was trained to fill form fields with fixed input values and click buttons. However, the performance of this model was limited by a fixed set of input data and the imprecise detection of successful form submission. This paper proposes a model to address these limitations. First, we use a large language model with data fakers to generate a wide variety of input data. Additionally, whether form submission is successful is partially determined by GPT-4o. Experiments show that our method increases average statement coverage by 2.3% over the previous model and 7.7% to 11.9% compared to QExplore, highlighting its effectiveness.

Keywords: large language model; prompt tuning; web app testing; web crawler; GPT-4o

check for updates

Academic Editors: Kaushik Roy and Valerie L. Shalin

Received: 16 December 2024
Revised: 18 January 2025
Accepted: 31 January 2025
Published: 5 February 2025

Citation: Chen, F.-K.; Liu, C.-H.; You, S.D. Using Large Language Model to Fill in Web Forms to Support Automated Web Application Testing. *Information* **2025**, *16*, 102. https://doi.org/10.3390/info16020102

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the rise of the Internet, many services are now offered as web applications. As of 2021, there were 1.88 billion websites, and approximately 79% of companies worldwide have their own websites [1]. Web applications vary widely in structure and functionality, depending on their purposes. A simple webpage might be a survey with only a few button elements, while a highly complex application could include registration forms, multi-step forms, and interactive forms with dynamic content. Since each website requires testing before going live and further testing with every update, the challenge of how to effectively test web applications has become a critical issue for these companies.

Broadly speaking, web application testing can be performed manually or automatically. Manual testing involves human testers interacting with the web application, such as filling out forms, to identify potential software defects. These testers need extensive software testing knowledge and a thorough understanding of the application. The advantage of manual testing is its ability to test specific scenarios and catch detailed issues, such as identifying invalid input values for web forms. However, manual testing is time-consuming and costly, especially for large and complex applications.

Automated testing, on the other hand, does not require human testers. Using web crawlers, the testing platform can explore web applications, automatically fill out encountered forms, and generate visual state diagrams for analysis. This approach reduces the manual workload, making it especially attractive for large-scale or frequently updated applications.

However, automated testing also faces some challenges during implementation. When web crawlers encounter forms that require input values, the platform needs to provide suitable input values for each field. There are usually two methods to generate these input values [2]. The first method is to use a "Monkey" approach, which involves randomly selecting predefined input data. This method is relatively simple and requires testers only to set up a set of input values. However, since the Monkey approach lacks knowledge about the form fields, its random trials can be extremely slow. The second method involves using an agent to select input values, either predefined or artificially generated. This method is more efficient but much more difficult to implement, especially in making the agent smart enough to understand the content of the fields to be filled.

Previously, we proposed the modified Using Agents to Automatically Choose Input Data (mUSAGI) model, which uses reinforcement learning to train an agent for automated testing of web applications (cf. Section 3 for a more detailed description) [3]. While mUSAGI can successfully test web applications after training, it has several limitations. One limitation is that the types of input values are fixed to six types, with one value per type. Another limitation is that the determination of whether a form is successfully submitted relies solely on page comparison results, which sometimes leads to misclassification. To address these issues, we made the following modifications:

1. We use Google's T5 (text-to-text transfer transformer) model [4] with prompt tuning [5,6] to classify the category of a field. Then, we use a Mocker [7] to generate the value for the field. This approach ensures that the input values are not confined to predefined values.
2. To improve the accuracy of determining whether a form is successfully submitted, we use GPT-4o [8] to assist in the decision-making process, in addition to page comparison.

The rest of the paper is organized as follows. Section 2 briefly reviews related work. Section 3 describes the mUSAGI approach. Section 4 details the proposed approach. Section 5 covers the experiments and results. Finally, Section 6 presents the conclusion and future work.

2. Related Work

Wang et al. [9] introduced an efficient method for automatically generating linearly independent paths in white-box testing. Their approach involves transforming the source code into a strongly connected graph and then applying an algorithm to identify these paths. Please note that the method discussed in this paper does not require any specific analysis of the source code prior to testing.

Malhotra et al. [10] proposed a method for automatically filling out web forms based on Bayesian inference. This method selects field values by generating instance templates and checking their informativeness. It uses Bayesian networks for value selection to improve prediction accuracy and computational efficiency. These templates are applied to the filling of multi-attribute forms. The method was tested using the multi-field search interface on the Runeberg.org website. Experimental results show that this method outperforms the existing Term Frequency–Inverse Document Frequency (TF-IDF) method in terms of accuracy, discrimination, and computation time. It effectively extracts deep web data, reduces the number of form submissions, and improves the efficiency and effectiveness of data retrieval.

Sunman et al. [11] proposed a semi-automated method and tool called AWET, which combines exploratory testing (ET) with crawler-based automated testing for web application testing. This tool involves manually recording a set of test cases through ET beforehand. These test cases are then used as a basis for exploring and generating test cases for the web

application. Experimental results show that AWET significantly outperforms the existing Crawljax [12] tool in terms of test coverage on five different web applications. Additionally, it can complete the exploratory test recording within 15 min and significantly reduce the overall testing time.

Crawljax is a widely utilized crawler that serves as a foundational tool for many researchers. For example, Negard and Stroul [13] created an intuitive, human-readable scripting language for Crawljax, designed to describe user testing scenarios and enhance automated testing. Wu et al. [14] expanded the capabilities of Crawljax by enabling it to remember user profiles associated with inputs for future use. Additionally, they conducted perturbation on the stored profile data to assess how an application under test can identify illegal input data.

Groce [15] utilized an adaptation-based programming (ABP) approach that incorporates reinforcement learning to automatically generate test inputs. This method involves calling the ABP library to generate test inputs for a Java program under test (PUT), aiming to uncover new behaviors of the PUT and optimize rewards based on increased test coverage. The experimental results show that this approach is highly competitive compared to random testing and shape abstraction techniques for testing container classes.

Lin et al. [16] introduced a natural-language method for testing web applications using crawling techniques. This method involves extracting and representing the attributes of a Document Object Model (DOM) [17] element and its nearby labels as a vector. This vector is then converted into a multi-dimensional real-number vector through various natural-language processing algorithms, such as bag-of-words. By analyzing the semantic similarity between the training corpus and the transformed vector, the method identifies an input topic for the DOM element. The input value for the element is then selected from a pre-established databank based on the identified topic. Experimental results indicate that this approach performs as well as or better than traditional rule-based techniques.

Qi et al. [18] proposed a keyword-guided exploration strategy for testing web pages, which achieves higher functionality coverage than generic exploration strategies. However, this approach is not fully automated as it requires a predefined set of keywords.

Liu et al. [19] introduced a method known as GUIDE for testing web applications using user directions. GUIDE prompts the user for input when encountering input fields on web pages. Test results show that GUIDE can discover more code compared to traditional web crawlers, but it still relies on human intervention for providing inputs. This research seeks to employ reinforcement learning to train an agent that can provide inputs autonomously, aiding web crawlers in discovering more code.

Carino and Andrews [20] introduced a method for testing application GUIs using ant colony optimization (ACO). Their approach, named AntQ, integrates an ant colony algorithm with Q-learning. AntQ generates event sequences that navigate through the GUIs and utilizes the resulting state changes as objectives. Test results demonstrate that AntQ surpasses random testing and conventional ant colony algorithms in identifying statements and faults.

Nguyen and Maag [21] used a support vector machine (SVM) to detect the search bar (or search function) in web pages and perform testing on the searching functionality to achieve the goal of codeless testing. As a web page has a wide variety of elements and functions, their approach has only limited usage.

Kim et al. [22] proposed a method that uses reinforcement learning to replace human-designed metaheuristic algorithms in search-based software testing (SBST). SBST algorithms seek to generate optimal test data based on feedback from a fitness function. They used the double deep Q-networks (DDQN) algorithm to train a reinforcement learning agent. The

reward is computed based on a fitness function. Experimental results demonstrate that this approach is effective for training functions written in C.

Zheng et al. [23] introduced WebExplor, an automated testing method for web applications that uses a reward function and deterministic finite automaton (DFA) to explore new web pages. The DFA tracks visited states, and if no new states are found, WebExplor selects a path and continues exploring. Tests show that WebExplor identifies more faults and operates faster than newer techniques. However, its use of Q-learning, based on application states, limits knowledge transfer between different applications. In contrast, the approach discussed here employs reinforcement learning to enable knowledge transfer from one application under test (AUT) to another.

Sherin et al. [24] proposed an exploration strategy for dynamic web applications called QExplore. Inspired by Q-learning, this method systematically explores dynamic web applications by guiding the search process, reducing or eliminating the need for prior knowledge of the web application. Q-learning is a reinforcement learning method that learns the optimal strategy in an unknown environment through trial-and-error interactions. QExplore uses a reward function to guide the exploration process and constructs a state graph during exploration. Experimental results show that QExplore achieves higher coverage and more diverse DOM states compared to Crawljax and WebExplor. It also results in more crawl paths, error states, and different DOM states, demonstrating its superior performance in testing dynamic web applications.

Liu et al. [25] presented a reinforcement learning method for workflow-guided exploration, aimed at mitigating the overfitting issue when training a reinforcement learning (RL) agent for web-based tasks like booking flights. By emulating expert demonstrations, this method incorporates high-level workflows to constrain allowable actions at each time step, thereby pruning ineffective exploration paths. This enables the agent to identify sparse rewards more swiftly while avoiding overfitting. Experimental results demonstrate that this approach achieves a higher success rate and significantly enhances sample efficiency compared to existing methods.

Mridha et al. [26] conducted a comprehensive literature review of automated web testing over the past decade, examining 26 recently published papers. The reviewed approaches are broadly categorized into model-based and model-free strategies. In model-free strategies, crawlers are generally employed to interact with the AUT, executing actions on encountered web pages. Notably, none of the reviewed papers incorporated any language models.

Liu et al. [27] proposed a method called QTypist, applied to graphical user interface (GUI) testing in Android. By using a pre-trained large language model to automatically generate semantic input text, it enhances the coverage and effectiveness of mobile GUI testing. Experimental results show that QTypist achieved a pass rate of 87% on 106 applications, which is 93% higher than the best baseline method.

3. Review of mUSAGI Method

Figure 1 provides an overview of mUSAGI, integrating the functions of both a crawler and an agent. In this model, the crawler is Crawljax (version: v3.7), and the agent is a feedforward network trained using the DQN algorithm [28]. The operating principle involves using the crawler to thoroughly explore web applications, identifying and collecting all input pages, which are pages containing forms. These collected input pages are then passed to the agent through the learning pool. The agent interacts with the forms available in the learning pool and then passes the results back to the crawler. This process enables the crawler to automatically perform appropriate actions based on different input pages and construct a directive tree.

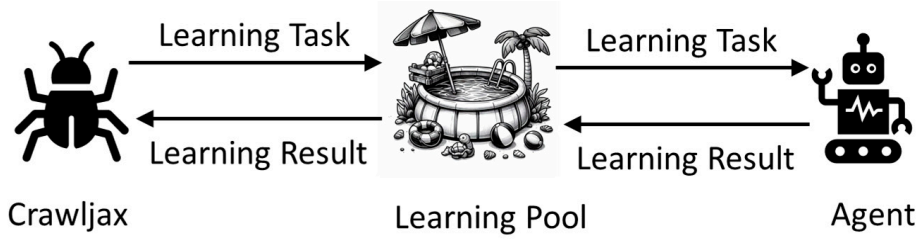

Figure 1. The overview of mUSAGI.

The mUSAGI model leverages the open-source crawler Crawljax for interacting with dynamic webpages. According to the Crawljax website [12], "Crawljax can explore any (even single-page dynamic JavaScript-based) web application through an event-driven dynamic crawling engine". Therefore, we believe this tool should be sufficient for our implementation. In fact, it is common to use Crawljax as a building block for conducting experiments [13,14].

In mUSAGI, the agent is trained using a reinforcement learning algorithm. Unlike the RL models reviewed in Section 2, which use the same AUT for training and testing, the mUSAGI model is trained on one AUT and tested on different AUTs. The overall process includes three main steps: collection, training, and testing, briefly described below.

1. Collection: In this step, we aim to gather as many input pages as possible. Each input page serves as an example for the agent to learn from. When encountering an input page, we use random actions (Monkey) to determine the values for input fields (e.g., Email, Name, Password) and then click the "submit" button.
2. Training: Using the input pages collected in the previous step, we train an agent with reinforcement learning, defining specific rewards. The agent's environment provides tags and texts of the fields. The actions involve selecting values to fill a form field from a list. Rewards are computed based on whether the agent selects the correct action according to the example. The training algorithm used is deep Q-learning [28]. The trained model is then stored.
3. Testing: In this step, we use the trained agent to test another web application, referred to as the AUT. The training and testing applications are different, allowing for a certain level of generalization. If the Istanbul middleware [29] supports the AUT, the model reports code coverage and a directive tree. Otherwise, only the directive tree is reported.

The structure of the directive tree includes root nodes, directives, and input pages, as shown in Figure 2. The directives on each path of the tree consist of a sequence of actions (values filled and buttons clicked, etc.) that enable the crawler to navigate from the home page to different target pages. The directive tree can be used to calculate the number of input pages, input page depth, and input page coverage (ICI) breadth. The root node is a virtual node that does not contain specific information and only connects to the initial page crawled by the crawler. Directive nodes record the current usage scenarios and corresponding operations explored by the agent. During the crawling process, if the same input page is encountered again, the crawler will perform interactive operations based on the action sequences in the directive nodes. Input page nodes record detailed information about the pages.

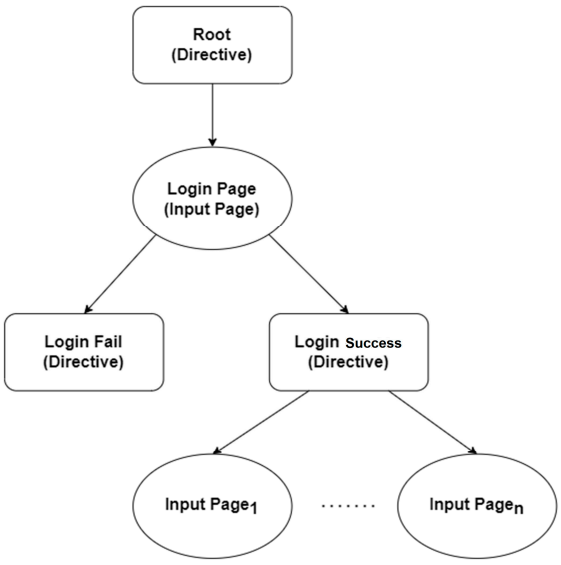

Figure 2. Illustration of a directive tree.

The mUSAGI model was successful but had some limitations, listed as follows:

1. Lack of diverse input data: In this model, the actions for filling fields are limited to the following values: Email, Number, Password, Random String, Date, and Full Name. With such a limited set of values, other types of fields may not be correctly filled. This lack of diversity in input values needs to be addressed. This is particularly important in testing web applications, where almost every AUT contains multiple forms. Enhancing the diversity of input data can ensure that software testing covers more forms, thereby improving the AUT's reliability and stability.
2. Long training time: In our previous method, Monkey was used to randomly fill form fields and collect web forms for training the agent. However, the agent initially requires many attempts to guess the correct field value, resulting in considerable time spent collecting training samples (forms). This paper proposes a different approach to reduce the training time.
3. Imprecise determination of form submission status. The previous method relied on DOM similarity to determine if a web form was successfully submitted. If the similarity between the postsubmission page and stored pages was less than 95%, the form was considered successfully submitted; otherwise, it was deemed a failure. The threshold of 95% was experimentally determined [3]. However, some web applications only display a small piece of confirmation text upon successful submission. In such cases, the overall DOM similarity remains very high, possibly over 95%, leading to false negatives where successful submissions are incorrectly marked as failures. This mistake causes the agent to repeatedly test the same successful page, lowering efficiency. Therefore, more reliable methods are needed to accurately determine form submission status to improve efficiency.

4. Proposed Approach

Figure 3 provides an overview of the method proposed in this paper. We adopt the same modular structure as mUSAGI but replace the agent with FormAgent and switch from reinforcement learning to a large language model (LLM). This change enhances the system's flexibility and efficiency, leveraging the advantages of LLMs in natural language processing to handle complex web structures and input data.

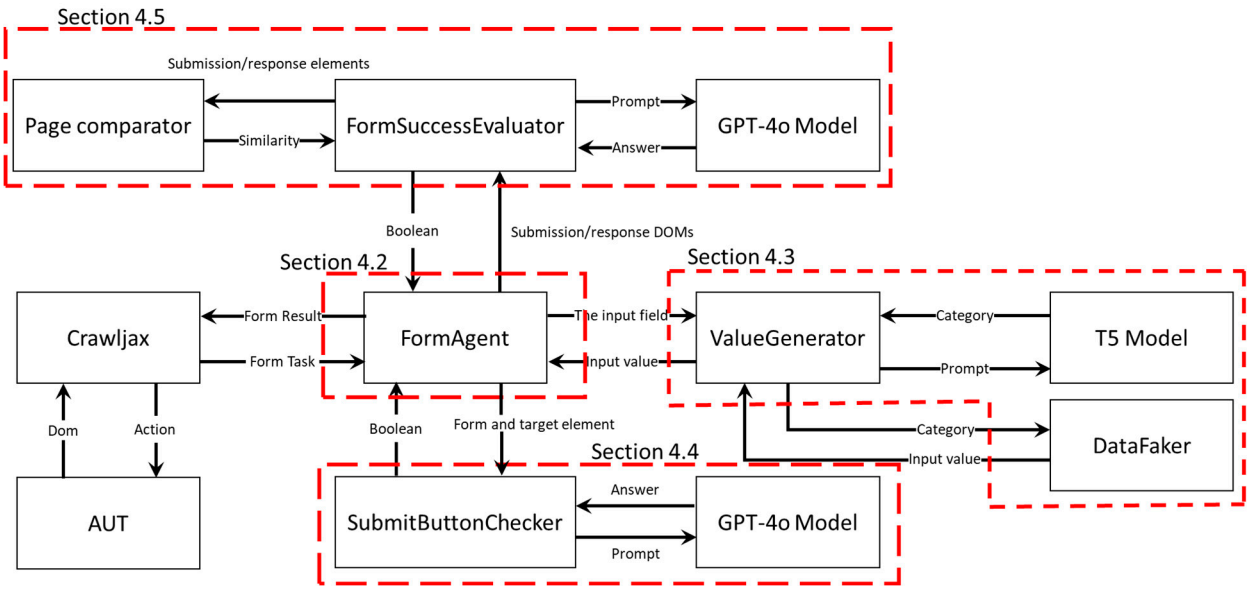

Figure 3. The proposed approach. The section to describe each box is given.

4.1. Overview of Proposed Approach

First, Crawljax crawls the web pages under test. During this process, Crawljax not only traverses the web pages but also analyzes and records the state of each page in detail. When the crawling algorithm detects that all states of the web application have been traversed and processed, Crawljax saves all pages containing forms (i.e., input pages).

Next, these input pages are sent to FormAgent. The main task of FormAgent is to analyze and process the input elements within these input pages. These input elements contain rich structured information, which is crucial for the subsequent classification and input data generation process.

After extracting the input elements from the input pages, FormAgent passes this information to the ValueGenerator. The ValueGenerator uses the powerful natural language processing capabilities of the T5 model [4] to classify these input elements, determining the category of each input element, such as email address, phone number, etc. This classification process makes the subsequent value generation easier.

Once the T5 Model completes the task, the category information is handed over to DataFaker, using the open-source Mocker [7]. DataFaker's task is to generate an appropriate input value based on the category of the field. For instance, for an email address field, DataFaker generates a correctly formatted email address. These automatically generated data are then used to interact with the encountered form and to construct a portion of a directive.

Finally, FormAgent checks these directives to ensure their validity. This includes verifying whether buttons are clicked, all input fields have data entered, and whether the directives can successfully submit the form. This validation ensures that the directives can be correctly executed in practice. If the directives are confirmed to be valid, they are handed back to the crawler to further search for deeper input pages, ensuring comprehensive coverage and testing of the web application.

4.2. FormAgent

The FormAgent determines whether a form should be interacted with and if the encountered form is new. For a new form, the ValueGenerator is used. For a previously interacted form, stored values are used. To determine if a webpage contains a form, we scan the entire DOM structure to identify elements with the HTML tag "input". We then

filter out elements whose parent is a "form" element to accurately identify form structures on the webpage. If one or more qualifying elements are found, we classify the page as an "input page".

After confirming a webpage as an input page, the system compares it with the already discovered input page list to ensure uniqueness and avoid duplicates. If no duplicates are found, the page is added to the list. During the comparison process, special attention is given to variable elements, which are dynamic web elements that change over time. These elements must be removed before comparison to ensure accuracy.

4.3. ValueGenerator and Prompt Tuning

4.3.1. T5 Model

Gur et al. investigated how well LLMs understand HTML [30]. They compared various models, including BERT [31], LaMDA [32], and T5 [4]. By fine tuning the T5 model with HTML data, they found that T5 excelled in tasks such as classifying HTML elements, generating descriptions, and navigating web pages. Specifically, the WebC-T5-3B model achieved 90.3% accuracy in the semantic classification of HTML elements, demonstrating strong performance. This model was chosen for its excellent performance and lower resource requirements, making it more practical for real-world applications. This choice underscores the balance between model performance and resource efficiency.

4.3.2. Predefined Categories

Since DataFaker needs to know the category of fake data (value for filling one field) to be generated, it is necessary to classify each field's category. To simplify training (prompt tuning), we collected 75 webpages from 20 websites listed on Statista [33] and counted the categories of the fields on these websites. Table 1 shows some results.

Table 1. The categories extracted from popular webpages.

Category	Count	Category	Count
First Name	19	Province	1
Last Name	20	Region	1
Email	18	Number	25
Gender	1	Country	1
String	32	Display Name	1
User Name	12	Address	11
Full Name	9	Suburb	3
Postal Code	8	Company Name	1
Store Name	1	Card Number	1
Phone Number	7	Expiration Date	1
Street Address	8	CVV	1
City	5	Date	6
State	1		

In this paper, input elements on web pages that match the categories in Table 1 are manually labeled. The resultant dataset is then used to prompt-tune the T5 model to predict categories (see Section 4.3.3). This setup ensures that the T5 model outputs only the predefined categories, enhancing the overall stability and reliability of the system. If an input element does not match any of the predefined categories, the T5 model may choose the category most closely related to the predefined ones, as it is limited to selecting only from the list. Therefore, the Mocker can still produce values for form filling.

Note that Table 1 does not include the "Password" category because passwords cannot be randomly generated. Randomly generating passwords could lead to login failures,

causing infinite loops or stalls. Therefore, this paper sets a fixed password to ensure smooth login and operation of the web application during testing and tuning.

4.3.3. ValueGenerator and Prompt Tuning of T5 Model

In this section, we will explain how to perform prompt tuning on the T5 model. First, we used the forms mentioned in Section 4.3.2 as the training data source. For each form's input elements, we extracted necessary information such as labels, placeholders, and names. This information serves as the basis for classification. We then manually defined categories for each input element based on this information, ultimately generating a structured JavaScript Object Notation (JSON) file, as shown in Figure 4.

```
{
  "name": "email-prevent",
  "placeholder": "Email",
  "label": "Not Found",
  "category": "email"
},
{
  "name": "email",
  "placeholder": "Email",
  "label": "Email",
  "category": "email"
},
```

Figure 4. The JSON file for prompt tuning.

After obtaining the pre-trained LLM, it is necessary to adapt the model to predict the defined categories. To achieve this, we can use either fine tuning or prompt tuning. Fine tuning involves modifying the language model's parameters by using a specific dataset to adjust its internal weights. This precise adjustment enables the model to tailor its outputs, making them suitable for a particular application. However, there are some downsides to using fine tuning.

First, as fine tuning essentially involves training, all the weights in the pre-trained model need to be adjusted, as shown on the left side of Figure 5. This leads to longer training times and higher computing costs [34]. Second, the fine-tuning process must use a lower learning rate to avoid overwriting the pre-learned features too quickly [35]; otherwise, catastrophic forgetting might occur. Additionally, fine-tuned models tend to be less robust, as the model is retrained to be tailored to a specific application.

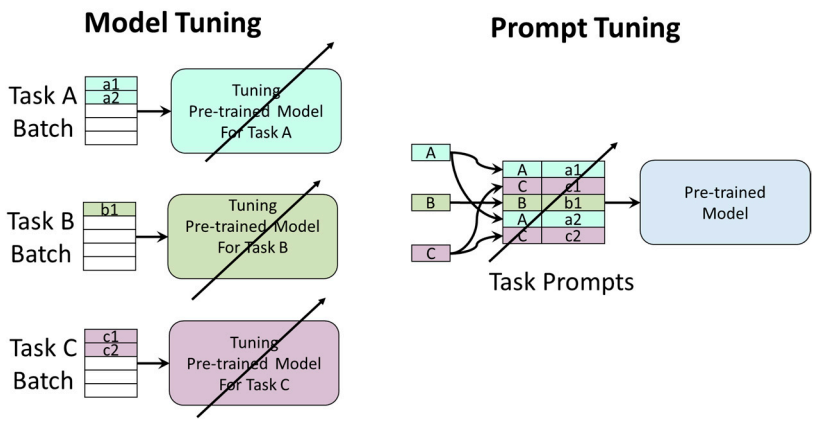

Figure 5. Difference between model fine tuning (**left**) and prompt tuning (**right**). The slash line with arrow indicates (fine) tuning of parameters (weights).

Prompt tuning [5] is a technique that allows LLMs to generalize more easily to downstream tasks, as shown on the right side of Figure 5. Unlike fine tuning, prompt tuning freezes the parameters of the pre-trained model and trains a small model in front of it. This approach significantly reduces the number of trainable parameters for each downstream task, thereby lowering computing costs. Additionally, there are open-source tools available for prompt tuning [6]. Consequently, we utilize this technique in our implementation. Essentially, the ValueGenerator in Figure 3 represents this part of the model.

In our implementation, prompt tuning is accomplished using Open Prompt [6]. This framework is designed to simplify and facilitate prompt engineering for natural language processing tasks. Open Prompt includes the following components:

1. Template: A key element of learning, it provides prompts by wrapping the original text in text or software-encoded templates, usually containing context markers.
2. PromptModel: This component is used for training and inference. It includes a Pre-trained Language Model (PLM), a Template object, and an optional Verbalizer object. Users can combine these modules flexibly and design their interactions. The main goal is to allow training through a unified API without needing specific implementations for different PLMs, enabling more flexible usage.
3. PromptDataset: This component is used to load training data.

During the prompt-tuning phase, we used the prompts shown in Figure 6 with Open Prompt. The prompts include keywords such as placeholder, text_a, soft, and mask. The placeholder allows dynamic insertion of required data into the prompt, making it flexible to adjust the model's input content. Text_a is the name specified when inserting data into the program, helping to identify and replace text at specific positions. Soft indicates a soft prompt, followed by text intended to initialize the soft prompt, allowing the model to start learning from meaningful text rather than random vectors. This initialization strategy enables quicker convergence to the (sub)optimal state. After tuning, the model is used to determine the categories of input fields.

```
Prompt

The {"placeholder": "text_a"} label from web forms. {"soft": "The label belong to
category"} {"mask"}.
```

Figure 6. The used prompt to Open Prompt.

4.3.4. DataFaker

The DataFaker block utilizes the mocker-data-generator tool, which simplifies the creation of large amounts of mock data [7]. It employs schema-based fake data generators like FakerJs, ChanceJs, CasualJs, and RandExpJs to produce test data. This tool supports TypeScript types and can generate diverse data to meet various testing needs. Users can customize data models and combine multiple fake data generators to create complex data structures. Experimental results show that this tool can efficiently generate large amounts of data, aiding in data simulation during development and testing processes.

4.4. Submit Button Checker

In our previous work, one action was to "click" buttons. Through reinforcement learning, the agent learned when to click a submit button. In the current model, since reinforcement learning has been removed, determining whether the currently focused component is a submit button becomes an issue. Crawljax generates form tasks in a top-down manner according to the HTML structure. Without special arrangement, the form

agent will sequentially click button components from top to bottom after completing the form filling.

In some web applications, the first button component is not the submit button. For example, Figure 7 shows a form in the KeystoneJS web application. In Figure 7, there are three button components: Create, Cancel, and Close Window. However, the "x" button (close window in the top-right corner) appears at the top. Therefore, if the agent clicks buttons from top to bottom, it would click the close window button after completing the form filling. Closing this form causes a significant change in the screen, leading the agent to mistakenly judge the form as successfully submitted, while in reality, it has not been. The method for determining whether the form submission is successful can be found in Section 4.5.

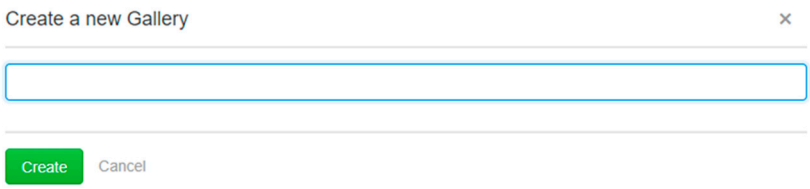

Figure 7. A form with three buttons.

To solve this problem, the proposed model uses GPT-4o [8] for support. Our method integrates the form element and the currently focused target element into our prompt to query GPT-4o. Specifically, we use a system prompt to guide GPT-4o on how to respond and the role it should assume, as shown below:

You are an AI web crawler assistant.

The user will give you some web elements.

Please answer if it is a form submission button.

Please say only yes or no.

After that, the web elements containing the button information are provided to GPT-4o, as shown in Figure 8. This way, whether the button is a submission button can be determined.

```
The Form element:
<form action="/submit" method="post">
    <label for="username">帳號:</label>
    <input type="text" id="username" name="username" required>

    <label for="password">密碼:</label>
    <input type="password" id="password" name="password" required>

    <button type="submit">確認</button>
</form>

The target element:
<button type="submit">確認</button>
```

Figure 8. The web elements sent to GPT-4o for decision.

4.5. Determination of Successful Submission

When testing web applications, it is crucial to determine if a submission is successful. If the submission fails, the application typically returns to a previously displayed page. From a software testing perspective, the code to render that page has already been examined. Therefore, testing new pages is always desirable. If the submission fails, the testing tool will attempt to fill out the same form again, using different values or clicking a different button. Conversely, if the submission is successful, a new page will appear, and a portion of new code will be tested.

To determine if a submission is successful, one possible method is to check the code coverage. However, the tool used to measure code coverage is language-dependent, meaning different programming languages require different tools. For example, the Istanbul middleware can measure code coverage for ES5 and ES2015+ JavaScript code but not for other languages like PHP or Python. To compute the coverage of PHP code, a different tool must be used. This situation is undesirable as it complicates the testing platform, making it difficult to accommodate applications written in different languages.

In our previous model (mUSAGI), we used an alternative approach. We employed a page comparison algorithm to determine if the page that appeared after clicking the submit button was similar to any previously encountered pages. If the similarity score was lower than a threshold, it indicated a new page had been encountered. This page was then stored, and the submission was deemed successful. Otherwise, the submission was considered unsuccessful, and the page was not stored. For a detailed description of this method, please refer to [3].

In the previous model, the tag, class, and text elements in the DOM structure were extracted for comparison, and the similarity threshold was set to 95%, resulting in the highest classification accuracy. Though effective, this method sometimes fails, especially when the screen only shows a single line of text indicating the success of the submission.

As shown in Figure 9, after the form is successfully submitted, the screen only displays a small segment of text with a green background, indicating to the user that the form was successfully filled out. Due to these very subtle changes, the similarity score is usually above 95%. This means that when the form is successfully submitted, the similarity remains high. Therefore, such high similarity cannot effectively distinguish whether the form was successfully submitted.

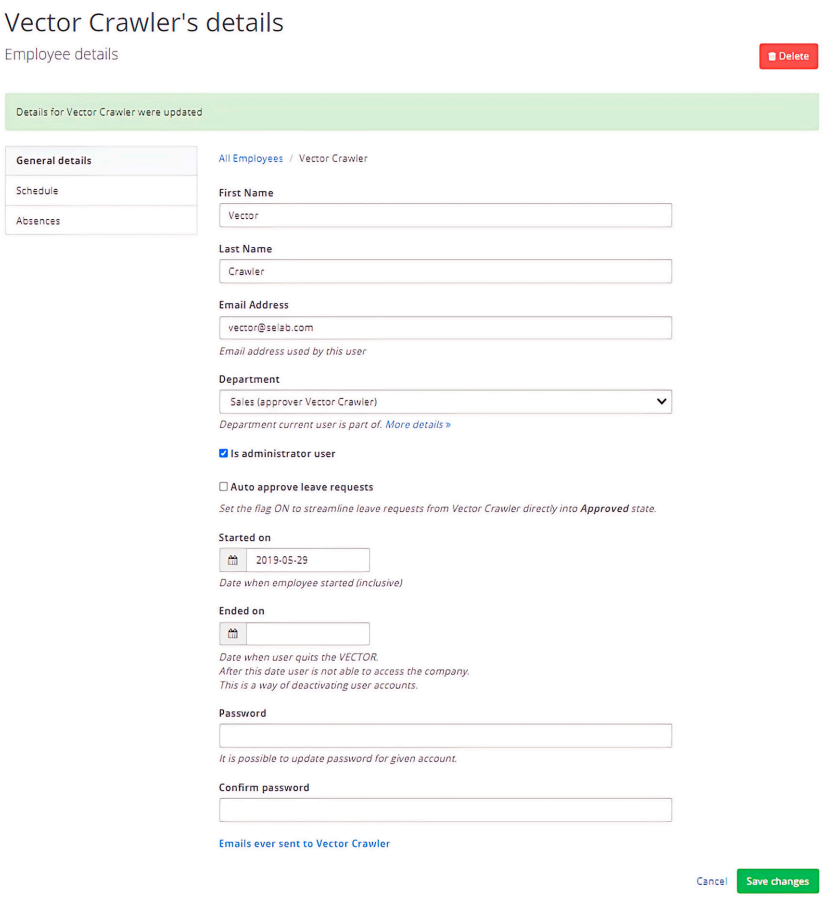

Figure 9. A web page after successful submission with minor change to page contents.

This paper proposes a new method that uses the GPT-4o model to assist in determining whether a form has been successfully submitted. To save time and budget, as GPT-4o is a paid service, the model is only utilized if the similarity score exceeds 95%.

In the original page comparison algorithm, the DOM elements (tag, class, and text) of both pages are compared. To use GPT-4o, the differing parts of the strings are then sent to GPT-4o, and through the system prompt, the possible answers are restricted to "Yes" or "No". This allows for a clear determination based on GPT-4o's response. The algorithm for this part is given in Algorithm 1.

Algorithm 1 Determination whether a directive is effective.

Algorithm 1: Is directive effective

```
Input: Page beforeSubmitPage, Page afterSubmitPage
Output: Boolean isSimilar
begin
similarity ← calculatePagesSimilarity(beforeSubmitDom, afterSubmitDom)
  if similarity == 100 then
    return false
  end if
  if similarity >= 95 then
    beforeSubmitElements ← getElements(beforeSubmitPage)
    afterSubmitElements ← getElements(afterSubmitPage)
    isSimilar ← getGptAnswer(beforeSubmitElements, afterSubmitElements)
    return isSimilar
  end if
return true
end

procedure getGptAnswer(beforeSubmitElements, afterSubmitElements)
begin
  differentElements ← getDiffElements(beforeSubmitElements,
afterSubmitElements)
  answer ← openAiApi(differentElements)
  if answer == "yes" then
    return true
  else if answer == "no" then
    return false
  end if
end
```

5. Experiments and Results

This section covers the experimental environment, performance metrics, experiments, and results. Three experiments were conducted for evaluation. The first experiment evaluates the usefulness of using an LLM (and DataFaker) to fill forms. The second experiment assesses the effectiveness of the proposed approach in detecting "click" buttons and successful submissions. The third experiment compares the performance of the proposed approach with other methods. Finally, there is a subsection discussing threats to validity and another one for future work.

5.1. Experimental Environment

There are several considerations for choosing AUTs. This research focuses on leveraging LLMs to automate web form filling, thereby facilitating web page exploration and testing. To avoid intervening in commercial websites, our AUTs are limited to open-source web applications with rich web forms. Additionally, to ease the comparison of code coverage with other methods, we chose some web applications developed with specific server-side technologies, such as Node.js. Furthermore, we have previously used some AUTs, and to save on the initial setup time, we decided to continue using these AUTs.

The experiments were conducted through computer simulations, with the hardware specifications listed in Table 2. The AUTs under test are as follows: TimeOff.Management (TimeOff) [36], NodeBB [37], KeystoneJS [38], Django Blog (Django) [39], and Spring Petclinic (Petclinic) [40], detailed in Table 3. Among the AUTs, the first three are written in Node.js, allowing us to obtain code coverage with Istanbul middleware [29]. However, the Istanbul middleware cannot compute the code coverage for Django Blog (written in Python) and Spring Petclinic (written in Java). We use these two applications to demonstrate that other metrics can also be used to assess the relative performance of various automated testing platforms.

Table 2. List of experimental hardware specifications and software versions.

Hardware/Software	Specifications/Model/Version
CPU	Intel Xeon W-2235 3.80 GHz
RAM	32 GB DDR4
OS	Ubuntu 20.04
GPU	NVIDIA GeForce RTX 2070 8 GB GDDR6
Selenium	3.141.0
Crawljax	3.7
Python	Version: 3.7

Table 3. List of AUTs.

Application Name	Version	GitHub Stars Count	Lines of Code	Type
TimeOff.Management [36]	V0.10.0	921	2698	Attendance Management System
NodeBB [37]	V1.12.2	14 k	7334	Online Forum
KeystoneJS [38]	V4.0.0-beta.5	1.1 k	5267	Blogging Software
Django Blog [39]	V1.0.0	26	-	Blogging Software
Spring Petclinic [40]	V2.6.0	22.9 k	-	Veterinary Client Management System

During testing, the testing engineer needs to download the AUT to the local computer. Testing an online site is not appropriate, as the behavior of the crawler resembles a cyberattack. If Istanbul is able to measure the code coverage of the AUT, it should also be placed in the same folder as the AUT. Then, the programs are wrapped by Docker. If the AUT requires specific values for certain fields, such as a user ID and password, these values can be specified in the proposed model, along with the path to the Docker containing the AUT. Once preparation is complete, execute the proposed model for automated testing.

After the model cannot find any more new pages, the testing engineer can stop the model and obtain the state graph (directive tree) from Crawljax to compute the proposed metrics. Alternatively, if the code coverage information is available, it can be accessed via a browser connected to the AUT.

5.2. Performance Metrics

The following items are used to measure the performance of various testing methods:

1. Code coverage. According to Brader et al., "Low coverage means that some of the logic of the code has not been tested. High coverage...nevertheless indicates that the likelihood of correct processing is good" [41]. Therefore, a method achieving a higher percentage of code coverage is considered better. There are two types of code coverage: statement coverage and branch coverage. In the experiments, only statement coverage is reported, as these two are highly correlated. The choice of a code coverage tool is dependent on the programming language in use, as mentioned in Section 4.5. To supplement the code coverage metric, we introduce three additional metrics: the number of input pages, input page depth, and ICI breadth, detailed below.
2. Number of input pages. This value is the number of forms found by an approach. In the directive tree shown in Figure 10, input pages are represented by blue nodes. As Figure 9 has four blue nodes, the number of input pages in this directive tree is 4.
3. Input page depth. This is the number of nodes on the longest path from the root node to the deepest input page node. In Figure 10, the longest path is from the root node through directive node (marked as a red circle) with ID 77eb5790 to the final input page with ID 1068395108. Therefore, the depth of this tree is 2. This value is used to measure the capability of an approach to explore forms hidden deeply within the web application.
4. ICI breadth. This is the number of input nodes containing extensions to directive nodes. According to Figure 10, there are three input page nodes, each connecting to directive nodes, so the ICI breadth in this example is 3. ICI breadth can be used to count the number of forms successfully submitted. In most cases, this metric is highly correlated with the number of input pages.

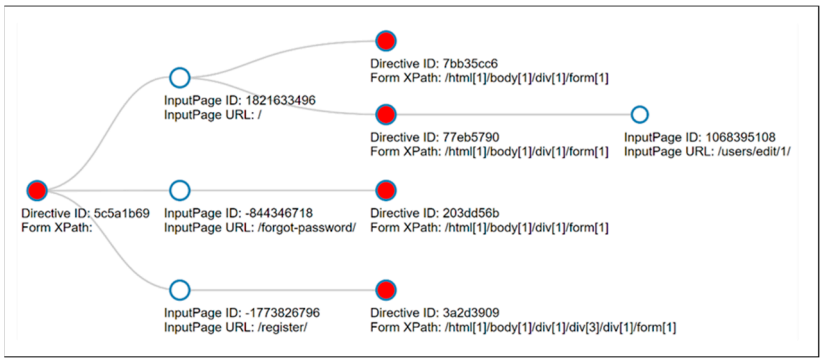

Figure 10. A directive tree.

An input node with a larger ICI breadth suggests that the input page may lead to a greater number of subsequent input pages, all of which should be explored and tested. A more effective crawler should generate diverse inputs to explore as many input pages as possible, as exploring more pages typically results in greater code coverage. Therefore, ICI breadth can be an indicator of the effectiveness of the inputs generated by the proposed approach.

5.3. Experiment One

This experiment compares the mUSAGI method with our method across five web applications. This experiment solely uses T5 and Mocker for form filling, excluding our proposed SubmitButtonChecker and FormSuccessEvaluator methods. Doing so ensures the evaluation of the capability of LLM on form filling. This model is called the T5 model. For clicking buttons and checking successful submissions, the mechanism in mUSAGI is still relied upon.

The experimental results are given in Table 4, showing values of code coverage, number of input pages, input page depth, and ICI breadth. It is worth noting that we slightly modified how to count the number of input pages in the experiments compared to the method used in mUSAGI. Therefore, the values reported here cannot be directly compared with the values provided in [3]. Note that the fractional number in the input page depth of the mUSAGI approach is due to the experiments being repeated three times to reduce the random fluctuations in performance caused by the use of the RL algorithm. There is no random fluctuation in the T5 model; therefore, the experiment was not repeated.

Table 4. Results for Experiment One.

Web App	Code Coverage		Input Pages		Input Page Depth		ICI Breadth	
	T5	**mUSAGI**	**T5**	**mUSAGI**	**T5**	**mUSAGI**	**T5**	**mUSAGI**
TimeOff	54.67	52.51	15	9	3	3	12	9
NodeBB	44.49	41.45	7	4	2	1.3	3	2
KeystoneJS	49.48	49.20	14	14	4	3.3	5	5
Django	-	-	6	4	3	2	6	4
Petclinic	-	-	17	14	9	3.3	9	9

As the AUTs differ, it is not possible to combine their results to compute statistical parameters, such as the standard deviation. For each individual AUT, although the mUSAGI tests each AUT three times, the standard deviation is not described in [3]. Recall that there is no random behavior in the proposed model. Therefore, only one set of values (such as code coverage and number of input pages) per AUT is obtained. Consequently, the experimental results are not sufficient to carry out meaningful statistical inferences. Hence, only average values are presented in Table 4.

It is observed that the code coverage of the T5 model outperforms the mUSAGI model in TimeOff, NodeBB, and KeystoneJS. When comparing the number of input pages in Table 4, it is obvious that the T5 model has higher values in the tested AUTs. With more input pages, it is natural to have higher code coverage. It is worth noting that although KeystoneJS has the same number of input pages when tested with both approaches, the T5 model has a higher input page depth. Therefore, it still has slightly higher code coverage. In this regard, both the "number of input pages" and "input page depth" values should be used for better assessment.

Overall, with the use of LLM for form filling, the performance improves on four of the five AUTs. The only AUT with minor improvement is KeystoneJS, which actually suffers from the problem of incorrectly detecting new pages, as discussed in Section 4.5.

5.4. Experiment Two

This experiment compares the mUSAGI method with our method across five web applications. Unlike Experiment One, this experiment includes the SubmitButtonChecker and FormSuccessEvaluator blocks. This model is called the T5-GPT model. As the T5 model shows minor improvement when testing KeystoneJS, this experiment will focus on

the observation of this AUT. For completeness, the results for the remaining AUTs are also provided.

The experimental results are shown in Table 5, where we observe that the T5-GPT model has a much higher number of input pages in KeystoneJS than the T5 and mUSAGI models (20 vs. 14). With successfully submitting more input pages, it is reasonable to assume that the T5-GPT model has higher code coverage in KeystoneJS. The value of code coverage in Table 5 confirms the assumption.

Table 5. Results for Experiment Two.

Web App	Code Coverage		Input Pages		Input Depth		ICI Breadth	
	T5-GPT	T5	T5-GPT	T5	T5-GPT	T5	T5-GPT	T5
TimeOff	54.74	54.67	14	15	3	3	14	12
NodeBB	44.52	44.49	7	7	2	2	3	3
KeystoneJS	50.86	49.48	20	14	4	4	5	5
Django	-	-	6	6	3	3	6	6
Petclinic	-	-	17	17	9	9	9	9

When testing the TimeOff app with both models, Table 5 shows that the T5-GPT model has a slightly lower number of input pages, the same amount of input page depth, and a higher amount of ICI breadth. The code coverage shows that the T5-GPT model has slightly higher code coverage. If comparing models only with the number of input pages and input page depth, we might conclude that the T5 model is slightly better than the T5-GPT model. However, it is actually not the case. Therefore, the ICI breadth is still a valid metric to measure the performance of the compared models.

Tables 4 and 5 indicate that the T5 model outperforms the original mUSAGI model, primarily due to superior form-filling efficiency. The mUSAGI model, trained on one AUT and tested on another one with 250 steps, struggles when fields in the tested AUT are absent in the training app, resulting in suboptimal action selection. Consequently, it is less efficient in form filling compared to the T5 model. Additionally, the T5-GPT model includes mechanisms to reduce false detection of successful submissions and button elements, further enhancing the efficiency of the proposed approach.

5.5. Experiment Three

This experiment compares the relative performance of the proposed T5-only, T5-GPT, mUSAGI, and QExplorer models. Although the source code of the Liu et al. model is available [27], it is not suitable for comparison with our approach. Their model is designed for mobile apps, which run on mobile devices, whereas our model is intended to test web apps, which are executed on servers to provide web services. Due to this distinction, we cannot use their model for comparison.

5.5.1. Code Coverage Comparison with QExplorer and mUSAGI

The source code of QExplorer is also available online. However, this code was an early version and had some compatibility issues when executing on our computers. We spent time managing the code to work, but we are unable to confirm if this version of the code was actually used in the literature [24].

As QExplorer is a standalone platform, we were reluctant to revise the code to report the number of input pages, input page depth, and ICI breadth. After reconsideration, we decided to report only the results of code coverage. Therefore, only TimeOff, NodeBB, and KeystoneJS were tested. Additionally, when using QExplorer to test NodeBB, the tool continuously clicked on external links, eventually causing the browser to run out of

memory and crash. Therefore, QExplorer does not have code coverage for NodeBB. The experimental results are shown in Figure 11. It is observed that the proposed T5-GPT model outperforms the mUSAGI model. The mUSAGI model also outperforms QExplorer.

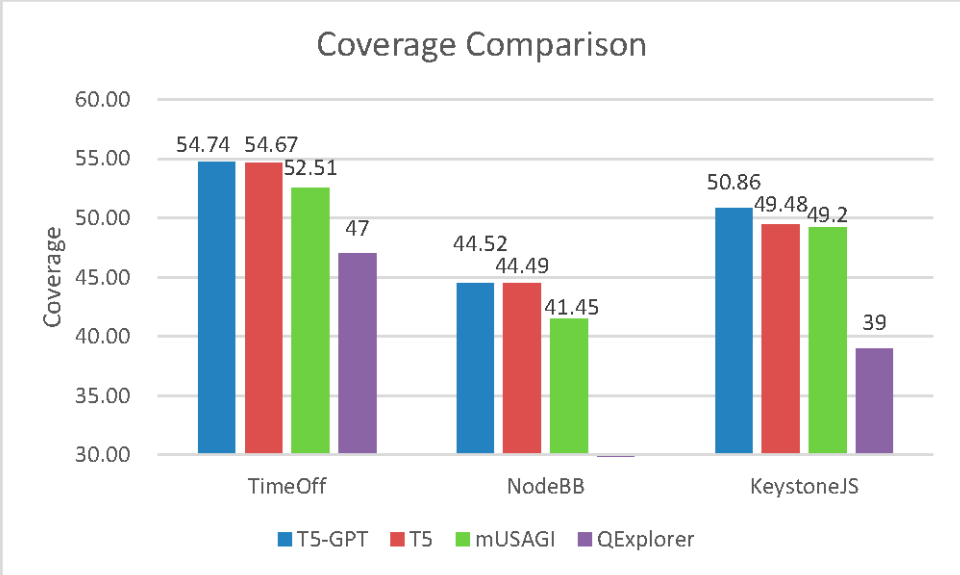

Figure 11. Comparison of code coverage.

5.5.2. Execution Time Comparison

In addition to code coverage, the execution times of mUSAGI, T5-GPT, and QExplorer are listed in Table 6. Since mUSAGI has both a training phase and a testing phase, it is difficult to directly compare its execution time with the others. Therefore, we provide both times. As [3] does not have an exact testing time, we use the rough estimate provided in the text. The execution time for QExplorer is set to 4 h, as extending the execution time does not improve code coverage.

Table 6. Execution time of various models (unit: hh:mm).

Web App	mUSAGI		T5-GPT	QExplorer
	Training	Test		
TimeOff	07:08	~01:00	08:09	04:00
NodeBB	42:19		05:11	-
KeystoneJS	20:45		05:28	04:00

It is important to note that the execution times of the proposed T5-GPT and mUSAGI models are mainly affected by the slow response of Crawljax. One interaction between Crawljax and the AUT can take several minutes. When analyzing the percentage of computing time, Crawljax accounts for 90.68%, GPT-4o for 1.31%, T5 for 0.63%, and the rest of the program for 7.38%. Therefore, it is apparent that Crawljax is the computing bottleneck. In contrast, QExplorer does not rely on Crawljax to interact with the AUT. Therefore, the comparison of execution times is only for reference and does not reflect the actual computational burden of the underlying algorithms of the studied approaches.

5.6. Threats to Validity

The threats to internal validity arise from the implementation of the platform and the manner in which the experiments were conducted. Firstly, the categories the T5 model can answer are limited to those given in Table 1, making it difficult to fill any field with an

untrained category, such as "PIN number," which has an exact number of digits (typically, six or eight). Next, because the values of the categories are generated from a Mocker, the proposed model is unable to generate all possible invalid inputs for comprehensive testing. Thirdly, the categories are not geographically sensitive. Therefore, it cannot handle categories whose formats depend on the selected locations. For example, the USA and UK have different postal code formats. These issues will be addressed in the next subsection.

The threats to external validity are related to the AUTs selected for the experiments. We use only five AUTs, which have a limited variety of field types in their forms. It is necessary to test more AUTs to determine if the observed experimental results can be generalized to other AUTs, and if the performance ranking of the studied models remains consistent.

5.7. Future Directions

In the future, we plan to improve the performance of the proposed T5-GPT model with the following points.

1. Enabling geographically sensitive values: The current model does not consider geography-related content from URLs or pages of the AUT. By applying LLMs to these elements, it is possible to store geographic information in the proposed model, enabling the generation of geographically sensitive values by LLMs.
2. Better mechanism to detect successful submission: The current implementation uses GPT-4o only if the similarity between two pages exceeds a threshold to lower the test cost since GPT-4o is not a free service. In the future, we plan to use existing pre-trained LLMs to detect successful submissions, thereby eliminating the need for a threshold.
3. Using LLM for value generation: Data fakers limit the categories of form fields. Instead, we can use an LLM as a value generator. To ensure the generated values are reasonable, another LLM will act as a verifier to approve the values from the generator.
4. Testing more AUTs: Currently, only five AUTs are used to evaluate the testing performance of the proposed model. Testing more AUTs from various categories, such as e-commerce and learning management systems, is desirable to better evaluate its usefulness.
5. Improvement of computational efficiency: The current model serves as a proof-of-concept, leveraging open-source resources like Crawljax, data fakers, and pre-trained LLMs. Consequently, its computational efficiency is not optimized. As mentioned in Section 5.5.2, Crawljax's execution time accounts for more than 90% of the total time. Due to its large codebase, improving Crawljax's response time is challenging. A potential future direction is to integrate the essential Crawljax functions into the proposed model to reduce overall execution time.

6. Conclusions

This paper introduces a method utilizing LLMs to enhance the efficiency of automated web application testing. Traditional reinforcement learning approaches rely on the learned agent to provide values for form fields and interact with buttons. In contrast, our method integrates the T5 model with prompt adjustments for value generation and uses GPT-4o for verifying successful submissions and identifying button elements. This approach effectively overcomes limitations of our previous model, such as limited input data diversity and inaccurate detection of successful submissions.

Experimental results confirm the superiority of the proposed method in terms of code coverage compared to our previous model, mUSAGI, and QExplorer. Our method increases average statement coverage by 2.3% over the previous model and 7.7% to 11.9% compared

to QExplorer. The experiments also show a high correlation between code coverage and proposed metrics, including the number of input pages, input page depth, and ICI breadth. Additionally, the system's modular approach, including components like FormAgent and ValueGenerator, facilitates future expansions and improvements.

In summary, this method offers an efficient solution for automated web application testing, with potential for widespread adoption in software development. Future work includes enabling geographically sensitive values, developing better mechanisms for detecting successful submissions, using LLMs for value generation, testing more AUTs, and improving computational efficiency.

Author Contributions: Conceptualization, F.-K.C., C.-H.L., and S.D.Y.; methodology, F.-K.C., C.-H.L., and S.D.Y.; software, F.-K.C.; validation, F.-K.C., C.-H.L., and S.D.Y.; formal analysis, S.D.Y.; investigation, F.-K.C., C.-H.L., and S.D.Y.; resources, C.-H.L., and S.D.Y.; data curation, F.-K.C.; writing—original draft preparation, S.D.Y.; writing—review and editing, C.-H.L.; visualization, F.-K.C.; supervision, C.-H.L., and S.D.Y.; project administration, C.-H.L., and S.D.Y.; funding acquisition, C.-H.L., and S.D.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by National Science and Technology Council, Taiwan, with grant number 112-2221-E-027 -049 -MY2 and the APC was waived by the journal editor.

Data Availability Statement: The conducted experiments required no additional data. The source code will be available publicly after the paper is accepted at https://github.com/ntutselab/rlenvforapp (accessed on 15 December 2024).

Conflicts of Interest: Feng-Kai Chen was employed by the company Qnap. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. How Many Websites Are There? Available online: https://www.statista.com/chart/19058/number-of-websites-online/ (accessed on 13 January 2025).
2. Liu, C.-H.; You, S.D.; Chiu, Y.-C. A Reinforcement Learning Approach to Guide Web Crawler to Explore Web Applications for Improving Code Coverage. *Electronics* **2024**, *13*, 427. [CrossRef]
3. Lai, C.-F.; Liu, C.-H.; You, S.D. Using Webpage Comparison Method for Automated Web Application Testing with Reinforcement Learning. *Int. J. Eng. Technol. Innov.* **2024**. *accepted*.
4. Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P.J. Exploring the Limits of Transfer Learning with a Unified Text-to-text Transformer. *J. Mach. Learn. Res.* **2020**, *21*, 1–67.
5. Lester, B.; Al-Rfou, R.; Constant, N. The Power of Scale for Parameter-efficient Prompt Tuning. *arXiv* **2021**, arXiv:2104.08691.
6. Ding, N.; Hu, S.; Zhao, W.; Chen, Y.; Liu, Z.; Zheng, H.-T.; Sun, M. Openprompt: An Open-Source Framework for Prompt-Learning. *arXiv* **2021**, arXiv:2111.01998.
7. Mocker-Data-Generator. Available online: https://github.com/danibram/mocker-data-generator (accessed on 23 May 2024).
8. Hello GPT-4o. Available online: https://openai.com/index/hello-gpt-4o/ (accessed on 23 May 2024).
9. Wang, X.; Jiang, Y.; Tian, W. An Efficient Method for Automatic Generation of Linearly Independent Paths in White-box Testing. *Int. J. Eng. Technol. Innov.* **2015**, *5*, 108–120.
10. Malhotra, D.; Bhatia, R.; Kumar, M. Automated Selection of Web Form Text Field Values Based on Bayesian Inferences. *Int. J. Inf. Retr. Res.* **2023**, *13*, 1–13. [CrossRef]
11. Sunman, N.; Soydan, Y.; Sözer, H. Automated Web Application Testing Driven by Pre-recorded Test Cases. *J. Syst. Softw.* **2022**, *193*, 111441. [CrossRef]
12. Crawljax. Available online: https://github.com/zaproxy/crawljax (accessed on 25 October 2023).
13. Negara, N.; Stroulia, E. Automated Acceptance Testing of Javascript Web Applications. In Proceedings of the 2012 19th Working Conference on Reverse Engineering, Kingston, ON, Canada, 15–18 October 2012.
14. Wu, C.Y.; Wang, F.; Weng, M.H.; Lin, J.W. Automated Testing of Web Applications with Text Input. In Proceedings of the 2015 IEEE International Conference on Progress in Informatics and Computing, Nanjing, China, 18–20 December 2015.
15. Groce, A. Coverage Rewarded: Test Input Generation via Adaptation-based Programming. In Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering, Lawrence, KS, USA, 6–10 November 2011.

16. Lin, J.-W.; Wang, F.; Chu, P. Using Semantic Similarity in Crawling-Based Web Application Testing. In Proceedings of the 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST), Tokyo, Japan, 13–17 March 2017.
17. Document Object Model (DOM) Technical Reports. Available online: https://www.w3.org/DOM/DOMTR (accessed on 20 October 2024).
18. Qi, X.F.; Hua, Y.L.; Wang, P.; Wang, Z.Y. Leveraging Keyword-guided Exploration to Build Test Models for Web Applications. *Inf. Softw. Technol.* **2019**, *111*, 110–119. [CrossRef]
19. Liu, C.-H.; Chen, W.-K.; Sun, C.-C. GUIDE: An Interactive and Incremental Approach for Crawling Web Applications. *J. Super Comput.* **2020**, *76*, 1562–1584. [CrossRef]
20. Carino, S.; Andrews, J.H. Dynamically Testing GUIs Using Ant Colony Optimization. In Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering, Lincoln, NE, USA, 9–13 November 2015.
21. Nguyen, D.P.; Maag, S. Codeless Web Testing Using Selenium and Machine Learning. In Proceedings of the 15th International Conference on Software Technologies, Online, 7–9 July 2020.
22. Kim, J.; Kwon, M.; Yoo, S. Generating Test Input with Deep Reinforcement Learning. In Proceedings of the IEEE/ACM 11th International Workshop on Search-Based Software Testing (SBST), Gothenburg, Sweden, 28–29 May 2018.
23. Zheng, Y.; Liu, Y.; Xie, X.; Liu, Y.; Ma, L.; Hao, J.; Liu, Y. Automatic Web Testing Using Curiosity-Driven Reinforcement Learning. In Proceedings of the 43rd International Conference on Software Engineering (ICSE), Online, 22–30 May 2021.
24. Sherin, S.; Muqeet, A.; Khan, M.U.; Iqbal, M.Z. QExplore: An Exploration Strategy for Dynamic Web Applications Using Guided Search. *J. Syst. Softw.* **2023**, *195*, 111512. [CrossRef]
25. Liu, E.Z.; Guu, K.; Pasupat, P.; Shi, T.; Liang, P. Reinforcement Learning on Web Interfaces Using Workflow-Guided Exploration. In Proceedings of the International Conference on Learning Representations, Vancouver, BC, Canada, 30 April–3 May 2018.
26. Mridha, N.F.; Joarder, M.A. Automated Web Testing Over the Last Decade: A Systematic Literature Review. *Syst. Lit. Rev. Meta-Analy J.* **2023**, *4*, 32–44. [CrossRef]
27. Liu, Z.; Chen, C.; Wang, J.; Che, X.; Huang, Y.; Hu, J.; Wang, Q. Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, VIC, Australia, 14–20 May 2023.
28. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; et al. Human-level Control through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. [CrossRef] [PubMed]
29. Istanbul. Available online: https://istanbul.js.org/ (accessed on 20 October 2023).
30. Gur, I.; Nachum, O.; Miao, Y.; Safdari, M.; Huang, A.; Chowdhery, A.; Narang, S.; Fiedel, N.; Faust, A. Understanding HTML with Large Language Models. *arXiv* **2022**, arXiv:2210.03945.
31. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2018**, arXiv:1810.04805.
32. Thoppilan, R.; De Freitas, D.; Hall, J.; Shazeer, N.; Kulshreshtha, A.; Cheng, H.-T.; Jin, A.; Bos, T.; Baker, L.; Du, Y.; et al. Lamda: Language Models for Dialog Applications. *arXiv* **2022**, arXiv:2201.08239.
33. Most Popular Websites Worldwide as of November 2023, by Unique Visitors. Available online: https://www.statista.com/statistics/1201889/most-visited-websites-worldwide-unique-visits/ (accessed on 15 January 2025).
34. Fine-Tuning, vs. Prompt Engineering: How To Customize Your AI LLM. Available online: https://community.ibm.com/community/user/ibmz-and-linuxone/blogs/philip-dsouza/2024/06/07/fine-tuning-vs-prompt-engineering-how-to-customize?form=MG0AV3&communityKey=200b84ba-972f-4f79-8148-21a723194f7f (accessed on 17 January 2025).
35. Prompt Tuning, vs. Fine-Tuning—Differences, Best Practices and Use Cases. Available online: https://nexla.com/ai-infrastructure/prompt-tuning-vs-fine-tuning/?form=MG0AV3 (accessed on 17 January 2025).
36. TimeOff.Management. Available online: https://github.com/timeoff-management/application (accessed on 20 October 2023).
37. NodeBB. Available online: https://github.com/NodeBB/NodeBB (accessed on 20 October 2023).
38. KeystoneJS. Available online: https://github.com/keystonejs/keystone (accessed on 25 January 2024).
39. Django Blog. Available online: https://github.com/reljicd/django-blog (accessed on 20 October 2023).
40. Spring PetClinic. Available online: https://github.com/spring-projects/spring-petclinic (accessed on 20 October 2023).
41. Brader, L.; Hilliker, H.; Wills, A. *Testing for Continuous Delivery with Visual Studio 2012*; Microsoft: Washington, DC, USA, 2013; p. 30.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.